AI-Powered Student Project Repository with Self-Service Originality and iThenticate Plagiarism Checking

Vedansh
Dept. of Computer Science and Engineering
MLR Institute of Technology
Hyderabad, India
23R21A05EX

Shyamala
Dept. of Computer Science and Engineering
MLR Institute of Technology
Hyderabad, India
23R21A05DT

Ankith
Dept. of Computer Science and Engineering
MLR Institute of Technology
Hyderabad, India
23R21A05EF

Roshan
Dept. of Computer Science and Engineering
MLR Institute of Technology
Hyderabad, India
23R21A05DL

Dr. D.B.K. Kamesh
Dept. of Computer Science and Engineering
MLR Institute of Technology
Hyderabad, India
Professor and Project Guide

***Abstract*—Students usually learn that their project idea repeats earlier work only when a faculty guide reviews the proposal, and every plagiarism check of a report must be run by faculty on the institution's licensed checker. This paper presents a self-service portal that removes both bottlenecks. After signing in with their institutional Google account, students run two checks themselves. The idea check ranks all earlier projects by Sentence-BERT cosine similarity to a draft title and abstract and returns an originality report in a fraction of a second. The document check sends a full report to the institution's iThenticate licence through the iThenticate v2 API and returns its similarity against internet and journal sources; the API key stays on the server, drafts are not added to the iThenticate repository, and a weekly quota protects the licence. Every report carries a code signed with HMAC-SHA256 that faculty or external reviewers can verify. The idea check was evaluated on a benchmark of 96 project descriptions from eight domains and 32 test proposals, half of which restate existing projects in new words. Sentence-BERT ranked the restated project first in all 16 cases, against 11 for TF-IDF, and at the deployed threshold of 0.55 it flagged restated proposals with precision 0.94 and recall 1.00. A complete check took 18 ms on average on a laptop CPU.**

***Index Terms*—semantic similarity, Sentence-BERT, plagiarism detection, iThenticate, project repository, academic integrity, single sign-on, verifiable reports**

I. Introduction

A computer science department approves hundreds of mini, micro and major project proposals each year. Records of earlier projects are usually spread across spreadsheets, printed reports and the memory of individual guides. When a student brings a new idea, the guide has to decide whether it repeats earlier work, and this decision is slow and inconsistent. As a result, the same few ideas, such as automatic plant watering or face-recognition attendance, return year after year under slightly different titles.

Students face two separate questions. At the proposal stage they need to know whether the *idea* has already been done in the institution, and the evidence is only a title and a short abstract. Two descriptions of the same idea often share very few words, so matching on exact terms is not enough. Later, they need to know whether their written report overlaps with published or online text. Our institution answers the second question with a licensed iThenticate account, but that account is held by faculty, so students must hand over every draft and wait for a score. Students therefore make many trips to faculty for a check they could run themselves if the licence were exposed to them safely.

Our department's existing Micro Projects website lists past projects as static pages that can only be filtered by exact words. This work turns it into a portal with the following contributions:

- a self-service originality check that students run on their own drafts, as often as they need, before meeting their guide;
- semantic comparison with Sentence-BERT embeddings, with a TF-IDF word-overlap score shown next to it for transparency;
- self-service full-document plagiarism checks through the institution's iThenticate licence, using the iThenticate v2 API with a server-held key, draft checks kept out of the repository, and a per-student quota;
- sign-in restricted to verified institutional Google accounts, with student and faculty roles and a faculty approval workflow;
- tamper-evident originality reports that anyone can check with a short code; and
- an evaluation on a 96-project benchmark comparing semantic and lexical matching.

Section II reviews related work, Section III describes the design, Section IV covers the implementation, Section V reports the evaluation and Section VI concludes the paper.

II. Related Work

A. Project Repositories and Duplicate Checks

Jumoke and Folashade [1] describe a web system that flags a proposed final-year topic when its keywords already occur in earlier titles. The approach is fast but misses ideas that are

phrased differently. The academic project information management system in [2] centralises student groups, supervisors and submissions, but it performs no similarity analysis, so finding duplicates is left to the supervisor.

B. Lexical and Semantic Similarity

TF-IDF term weighting [3] combined with cosine similarity is the standard baseline for comparing short texts. It has been used to screen student theses [4], where a submission is flagged once its similarity to a stored thesis crosses a threshold. Because TF-IDF only rewards shared terms, its scores fall sharply for paraphrased text, as surveys of plagiarism detection point out [5], [6]; the same issue motivates work on text reuse across languages [7]. Tools that add syntactic and semantic checks for teachers have also been reported [8], [9], although their accuracy depends on the language model they use.

Sentence-BERT [10] fine-tunes a siamese transformer so that sentences with similar meaning are mapped to nearby vectors that can be compared with cosine similarity. Embeddings are computed once per document, so a whole repository can be ranked with a single matrix product. MiniLM [11] compresses a large transformer into a small model that runs comfortably on a CPU, which suits a departmental server.

C. Commercial Plagiarism Services

Services such as iThenticate compare a full document with web pages, published journal articles and previously submitted work, and report the share of matching text. iThenticate v2 exposes this through a REST interface, the Turnitin Core API [12]: a client creates a submission, uploads the file, requests a similarity report and reads back an overall match percentage. The API is licensed to the institution and authenticated with an institutional key, which is why access is normally limited to faculty accounts.

D. Research Gap

TABLE I
COMPARISON WITH EXISTING APPROACHES

Approach	Central repo	Semantic	Student self-check	Internet plagiarism	Faculty workflow	Verifiable report
Keyword duplicate check [1]	Yes	No	n/r	No	n/r	n/r
Project information system [2]	Yes	No	No	No	Yes	n/r
TF-IDF thesis screening [4]	Yes	No	n/r	No	n/r	n/r
AI checker for teachers [8]	n/r	Partial	n/r	n/r	Yes	n/r
Faculty-run iThenticate [12]	No	n/a	No	Yes	No	No
Proposed portal	Yes	Yes	Yes	Yes	Yes	Yes

Table I summarises these approaches (n/r: not reported; n/a: not applicable). Repository systems do not check documents against the internet, and the licensed checker is not available to students directly. None of them lets students test a proposal or a report on their own, and none issues a report whose integrity a third party can verify. The portal described next closes these gaps while keeping the final decision with faculty.

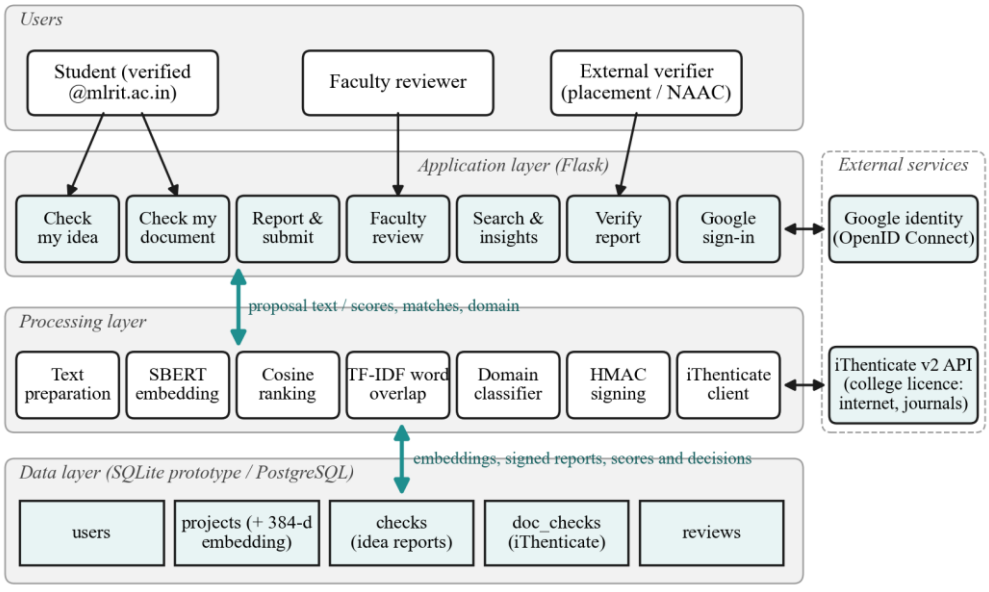


Fig. 1. Layered architecture of the portal. The application layer calls the processing layer with the proposal text or uploaded document; the processing layer talks to Google for sign-in and to the institution's iThenticate licence for document checks; the data layer stores users, projects with their embeddings, signed reports, document-check results and faculty decisions.

III. System Design

A. Users and Requirements

The portal serves three kinds of users. *Students* sign in, check drafts, submit a proposal and follow its status. *Faculty reviewers* see submitted proposals together with their reports and record a decision. *Verifiers*, such as placement officers or accreditation auditors, can confirm a report with its code and need no account. The design follows six requirements: only members of the institution may sign in; drafts remain private; an idea check finishes in well under a second; every score is explained by the projects or sources behind it; the institution's plagiarism licence is used without exposing its credentials; and the final decision is always made by faculty.

B. Architecture

Fig. 1 shows the four internal layers and the two external services. The application layer is a Flask web portal with modules for the idea check, the document check, reporting, faculty review, search, verification and sign-in. The processing layer prepares text, computes embeddings, ranks the repository, calculates TF-IDF overlap, predicts the domain, signs reports and calls the iThenticate API. The data layer holds five tables, described in Section IV.

C. Sign-In With Institutional Google Accounts

Authentication uses OpenID Connect [13] with Google as the identity provider, so the portal never stores passwords. The sign-in request carries the hd parameter to pre-select the college account. That parameter is only a hint, however, so access is enforced on the ID token that Google returns: the server accepts a user only if email_verified is true, the hd claim equals the institution's domain and the email address ends with that domain. New users receive the student role, and the faculty role comes from an administrator-maintained list.

D. Similarity Engine

A proposal consists of a title t and an abstract a. The engine forms the input x by writing the title twice followed by the abstract, so that the title is not outweighed by a long abstract. A Sentence-BERT encoder f maps x to a 384-dimensional unit vector. For every stored project i with embedding e_i, the similarity is

$$s_i = f(x) \cdot e_i \,/\, (\|f(x)\|\,\|e_i\|) \qquad (1)$$

Since all vectors are normalised, (1) reduces to a dot product, and scoring the whole repository is one matrix-vector multiplication. The five highest-scoring projects are returned, and the originality score is defined from the closest match:

$$O = 100 \times (1 - \max(0, \max_i s_i)) \qquad (2)$$

Similarities below 0.55 are labelled *Original*, values from 0.55 to 0.75 *Related*, and values of 0.75 or more *High overlap*. The 0.55 boundary comes from the benchmark in Section V and should be recalibrated once real institutional records are loaded. Next to each semantic score, the report also shows a TF-IDF cosine score computed on unigrams and bigrams with English stop words removed. Comparing the two helps faculty distinguish copied wording from a similar idea expressed in new words. A student's own earlier submissions are excluded from the comparison, but proposals still pending from other teams are included, so two teams proposing the same idea in one semester are detected.

E. Domain Prediction

Each of the eight domains is described by a short list of characteristic phrases, for example “sensors, microcontroller, Arduino” for the Internet of Things. These descriptions are embedded once, and a proposal is assigned to the domain whose description vector is closest to it. No labelled training data is needed, and a new domain can be added simply by writing its description.

F. Full-Document Plagiarism Check Through iThenticate

The idea check answers whether a topic already exists in the institution; it cannot tell whether the written report copies text from the internet or from journals. For that, the portal acts as a gateway to the institution's iThenticate licence. A student uploads a PDF or Word file on the *Check my document* page and accepts the iThenticate end-user licence agreement, which the API requires for every submitter. The server then drives the iThenticate v2 API on the student's behalf, as shown in the sequence diagram below.

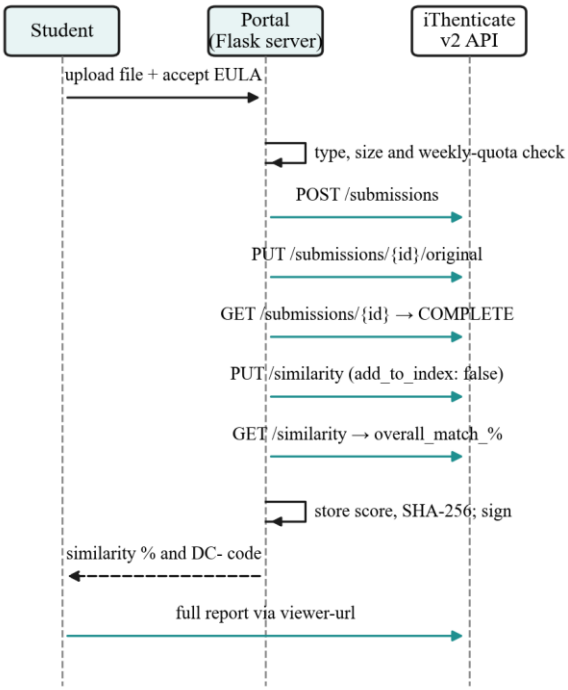


Fig. 2. Document check sequence. The portal calls the iThenticate v2 API with the institution's key; the student only sees the score, the report code and the full report viewer.

In Fig. 2, the server creates a submission owned by a pseudonymous identifier derived from the student's account, uploads the file, waits until text extraction is complete and requests a similarity report that searches the internet, published journals and submitted work, with quotes and bibliography excluded. It then reads the overall match percentage. Because these steps are asynchronous, the report page polls the API every few seconds until the score is ready.

The student can then open iThenticate's own viewer with the matched sources through a short-lived viewer URL.

Four design choices make this safe to open to every student:

- *Server-held key.* The institutional API key is stored only on the server; no faculty login or key is ever shared with students.
- *Draft checks are not indexed.* Each request sets add_to_index to false. If drafts were stored in the repository, the student's final submission would later match their own drafts and show a misleadingly high score.
- *Quota.* Each student may run a fixed number of document checks per week (three by default), which keeps usage within the licence.
- *Minimal retention.* The portal keeps the file's SHA-256 fingerprint and the score, not the file itself.

Faculty see a student's completed document checks next to the idea report on the review page, so the two results are judged together.

G. Verifiable Reports

Each idea check is stored under a code of the form OR-YYMM-XXXXXX, and each document check under DC-YYMM-XXXXXX; the last six hexadecimal digits are random. The server signs the stored report with a secret key K using HMAC [14]:

$$\sigma = \text{HMAC-SHA256}(K, \text{code} \parallel \text{content} \parallel \text{scores} \parallel \text{time}) \quad (3)$$

For an idea check, the content is the title and abstract, and the scores are the top similarity and the identifiers and scores of the matched projects; for a document check, the content is the file's SHA-256 fingerprint and the score is the iThenticate similarity. The public verification page recomputes σ; any later edit to the stored report causes a mismatch, and the page shows it as altered. The page also shows the faculty decision, which makes the report useful as evidence in placement interviews and accreditation audits.

H. Workflow

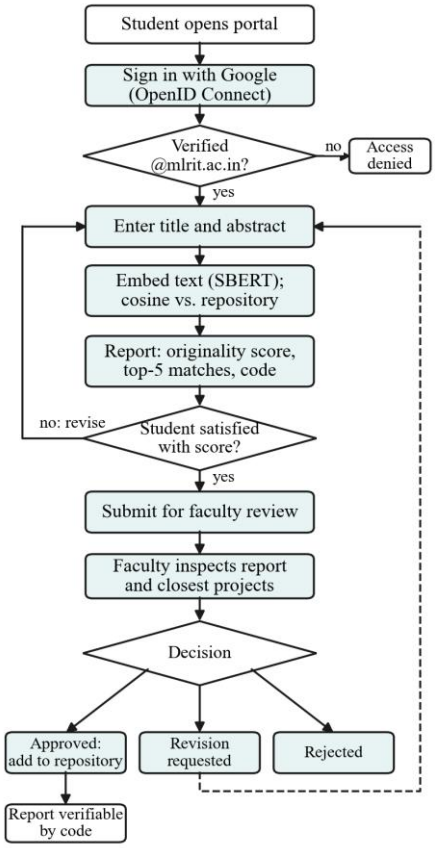


Fig. 3. Workflow from sign-in to approval. Students can revise and check again as often as needed before submitting; a revision request returns them to the editing step.

Fig. 3 shows the complete workflow. The loop between the report and the editing step is the key difference from faculty-only screening: a student who receives a low originality score can rework the idea immediately instead of waiting for a meeting. Approved proposals join the repository, so every new idea is checked against a record that grows from batch to batch.

IV. Implementation

A. Technology Stack

The prototype is written in Python 3.12 with Flask 3.1. Google sign-in is handled by the Authlib library. Embeddings come from the all-MiniLM-L6-v2 Sentence-BERT model (384 dimensions, about 22.7 million parameters) loaded through the sentence-transformers library, and TF-IDF scores come from scikit-learn 1.8 [15]. The prototype stores data in SQLite, with embeddings kept as 32-bit floating-point arrays. The same schema can be moved to PostgreSQL with the pgvector extension, which adds indexed nearest-neighbour search for large repositories. The interface uses Bootstrap 5 and the visual style of the existing Micro Projects website, so the portal extends the department's site instead of replacing it.

The iThenticate client is a small module built on the requests library. It sends the institutional key as a bearer token

together with the integration name and version headers that the API expects, and it calls the endpoints /submissions, /submissions/{id}/original, /submissions/{id}/similarity and /submissions/{id}/viewer-url under /api/v1. The institutional key had not yet been issued when this paper was written, so the client was tested against a local mock server that reproduces these endpoints and checks every request: the authorisation and integration headers, the end-user agreement version, the binary upload headers and the add_to_index flag. The full sequence of nine calls completed as expected. Until the key is configured, the portal runs the document check in a clearly labelled demo mode whose scores are simulated.

B. Data Model

TABLE II
DATABASE TABLES

Table	Main fields
users	email, name, role (student / faculty)
projects	title, abstract, domain, year, team, status, owner, report code, embedding
checks	code, user, title, abstract, top score, originality, band, domain, matches, signature
doc_checks	code, user, file name, SHA-256, iThenticate submission id, status, score, signature
reviews	project, faculty member, decision, comment, time

Table II lists the five tables. A draft lives only in the checks table; it is copied into projects with status *pending* when the student submits it, and its status later becomes *approved*, *revision* or *rejected*.

C. User Interface

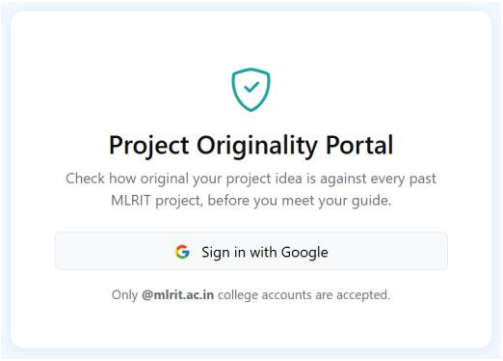


Fig. 4. Sign-in page. Only verified @mlrit.ac.in Google accounts are accepted.

Fig. 4 shows the sign-in page. After signing in, a student enters a title and abstract on the *Check my idea* page and is taken to the report.

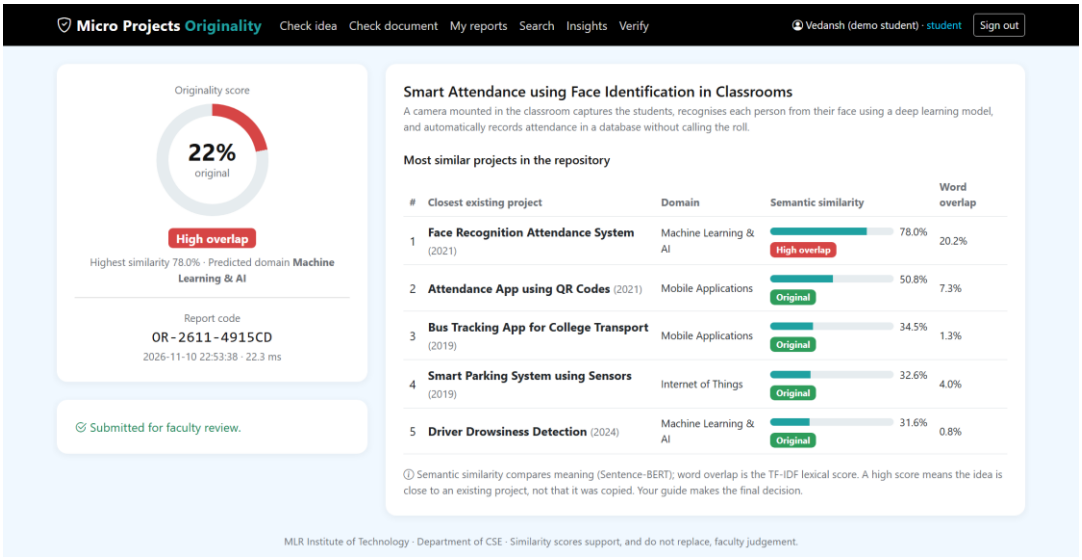


Fig. 5. Student originality report for a restated attendance proposal: 78.0% semantic similarity to an existing project but only 20.2% word overlap, giving 22% originality and a High-overlap label.

Fig. 5 shows a report for a proposal titled "Smart Attendance using Face Identification in Classrooms." Its closest match, "Face Recognition Attendance System," reaches 78.0% semantic similarity even though the two descriptions share only 20.2% of their weighted terms, so a keyword check would likely have missed it. When our own project proposal was checked, the closest match was an alumni networking portal at 37.4%, giving an originality score of 63% and the label *Original*.

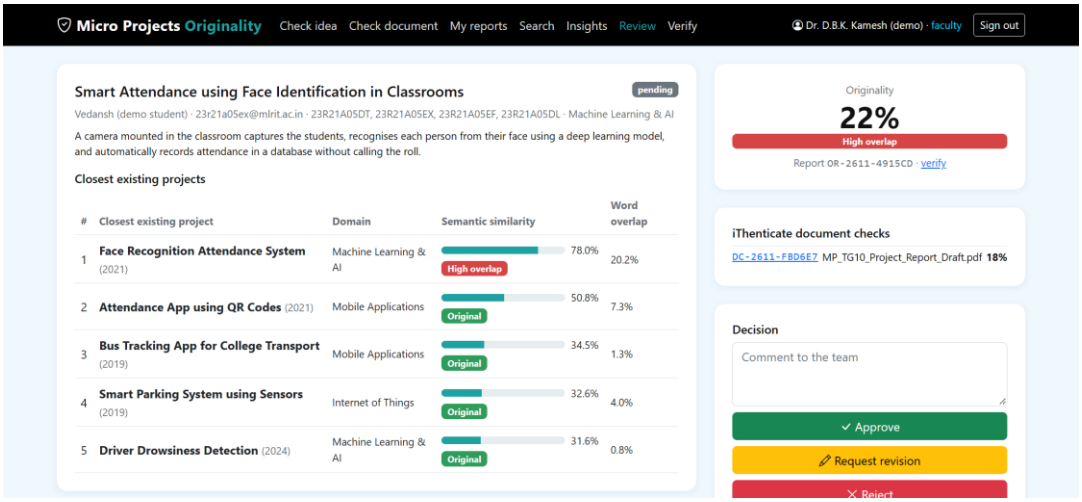


Fig. 6. Faculty review page with the proposal, its five closest projects and the decision controls.

Fig. 6 shows the faculty view of a submitted proposal. The reviewer sees the same evidence as the student, including the student's iThenticate document checks, and approves the proposal, rejects it or requests a revision with a comment; the decision is recorded with the reviewer's name and time.

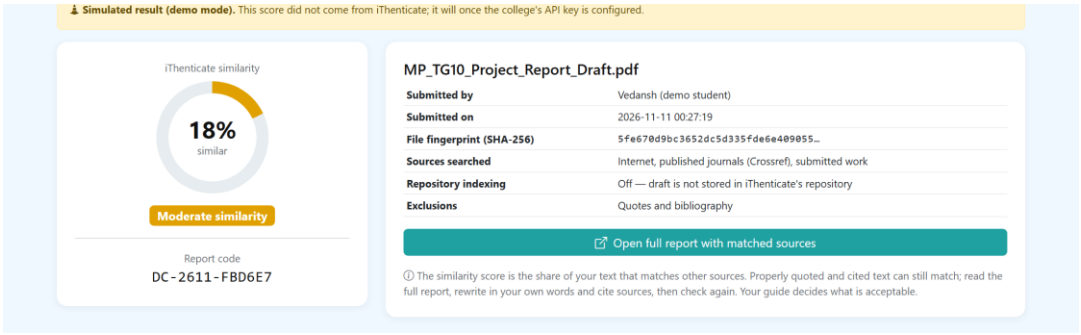


Fig. 7. Document check result page. The score shown here was produced in demo mode and is simulated; with the institutional API key, the same page shows the score returned by iThenticate.

Fig. 7 shows the result of a document check: the similarity percentage, the report code, the file fingerprint, the sources searched and the indexing setting, with a button that opens iThenticate's viewer for the matched sources.

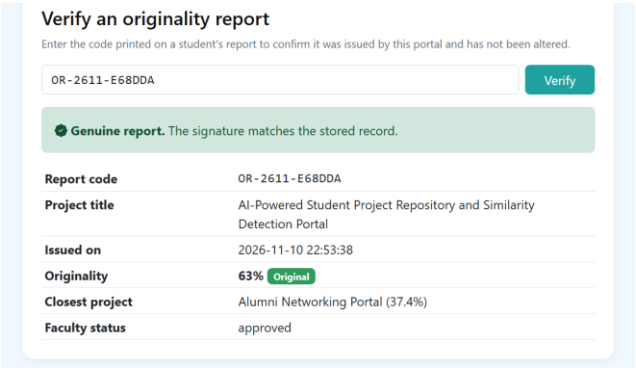


Fig. 8. Public verification of a report code, confirming the signature and the faculty decision.

Fig. 8 shows the verification page, which needs no sign-in. The portal also provides semantic search over approved projects. For example, the query “help blind people walk safely” returns the Smart Blind Stick project first. A topic-insights page counts projects per domain so that students can see which areas are crowded.

V. Experimental Evaluation

A. Benchmark

The repository contains 96 project descriptions, 12 in each of eight domains: machine learning and AI, web development, mobile applications, Internet of Things, blockchain, cybersecurity, data science and analytics, and cloud and DevOps. Eight descriptions come from projects already listed on the department's Micro Projects website; the authors wrote the remaining 88 to represent topics that are common in undergraduate projects. Because these are not official departmental records, the results show how the methods behave rather than the accuracy the system will reach on institutional data.

The test set has 32 proposals. In 16 of them, a repository project is rewritten with different vocabulary: for example, “Driver Drowsiness Detection” becomes “Alertness Monitoring System for Vehicle Drivers.” The other 16 are new ideas with no counterpart in the repository, but they fall in the same domains, so each has topically related neighbours.

B. Metrics and Setup

For the restated proposals, we measure how often the original project is ranked first (Hit@1) or among the top five (Hit@5), and the mean reciprocal rank (MRR). To measure flagging, each proposal is scored by its highest similarity, and we report the area under the ROC curve together with precision, recall and F1, both at the best threshold for each method and at the portal's fixed threshold of 0.55. Domain prediction is scored on all 96 repository records. Latency is the time for a complete check (embedding, semantic ranking, TF-IDF scoring and domain prediction) on an AMD Ryzen 7 6800H CPU with 16 GB of RAM and no GPU. The evaluation script is part of the project source code, so these results can be reproduced.

C. Results

TABLE III
RETRIEVAL AND FLAGGING RESULTS

Metric	TF-IDF + cosine	SBERT + cosine
Hit@1 (restated)	0.69	1.00
Hit@5 (restated)	0.88	1.00
Mean reciprocal rank	0.79	1.00
ROC-AUC	0.69	1.00
Best threshold	0.18	0.63
Precision / recall at best	0.68 / 0.81	1.00 / 1.00
F1 at best threshold	0.74	1.00
P / R / F1 at τ = 0.55	—	0.94 / 1.00 / 0.97
Mean top score: restated	0.25	0.67
Mean top score: new ideas	0.19	0.40

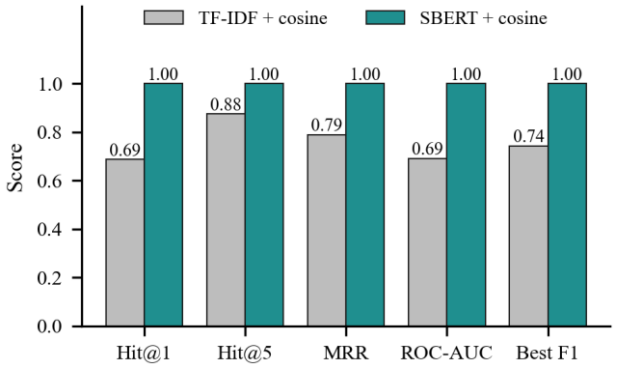


Fig. 9. Retrieval and flagging metrics for TF-IDF and Sentence-BERT.

Table III and Fig. 9 summarise the results. Sentence-BERT placed the original project first for all 16 restated proposals, while TF-IDF did so for 11 and missed the top five in 2. The ROC-AUC of TF-IDF (0.69) shows that word overlap barely separates restated proposals from new ones.

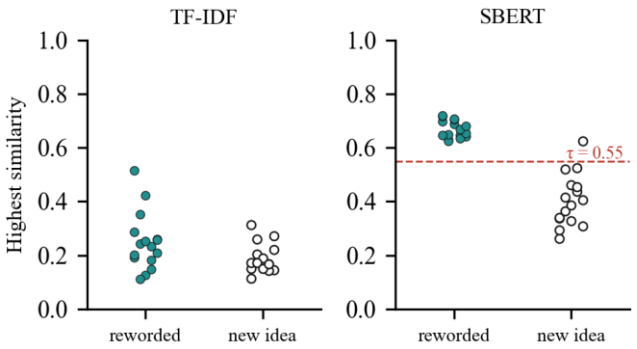


Fig. 10. Highest similarity of each test proposal. Filled points restate an existing project; hollow points are new ideas. The dashed line is the portal threshold.

Fig. 10 explains this result. Under TF-IDF, restated proposals average 0.25 and new ideas 0.19, a gap of only 0.06. Under Sentence-BERT the averages are 0.67 and 0.40, so the two groups barely overlap. At the fixed threshold, all restated proposals were flagged and only one new idea was flagged by mistake.

Domain prediction was correct for 87 of the 96 records (90.6%). Most errors fell between neighbouring areas, for example two machine-learning projects with a web front end that were labelled as web development. A complete check took 18.2 ms on average and 28.1 ms at the 95th percentile, far below the one-second target.

D. Discussion

The single false positive was “Sign Board Translation using OCR for Tourists,” which scored 0.63 against “Sign Language to Text Translator.” Both convert a visual sign into text, so the embedding places them close together even though the applications differ. This is the kind of case the *Related* band is meant for: the score asks faculty to look at the match rather than reject the proposal. Two other new ideas, a refreshable braille reader (0.52, nearest to the Smart Blind Stick) and a mood-tracking journal app (0.53, nearest to an expense tracker), fell just below the threshold, so borderline cases should be shown with their matches and not reduced to a single number.

The perfect best-threshold scores for Sentence-BERT are optimistic, because that threshold was chosen on the same data it was evaluated on. The results at the fixed threshold, precision 0.94 and recall 1.00, are a fairer estimate. The benchmark is also small and was written by the authors, and the restated proposals were written to change the vocabulary on purpose. Real student proposals will be noisier, so the thresholds must be recalibrated on departmental records.

E. Limitations

The document check has so far been exercised only against a mock of the iThenticate v2 API, because the institutional key was pending; its scores are produced by iThenticate itself, so the portal adds convenience and control rather than detection accuracy. API access must be enabled in the institution's iThenticate licence, and the weekly quota has to be agreed with the licence administrator. The idea check's model handles English only. Sign-in depends on the institution's Google

Workspace domain, and repositories with many thousands of projects will need the pgvector index mentioned in Section IV.

VI. Conclusion and Future Work

This paper presented a project repository portal that lets students run, on their own, the two checks they previously had to request from faculty: an idea check that uses Sentence-BERT similarity against all earlier projects, and a full-document plagiarism check that uses the institution's iThenticate licence through its v2 API without exposing the licence key. Sign-in is restricted to verified institutional accounts, the final decision stays with faculty, and every report can be verified by a short signed code. On a 96-project benchmark, semantic matching found every restated project that word matching often missed, and each idea check completed in tens of milliseconds on a laptop CPU.

Next steps are to connect the live iThenticate key and pilot the document check with one batch, to load the department's real project records and recalibrate the thresholds, to move storage to PostgreSQL with pgvector, to highlight the sentences that drive an idea match, to add support for Telugu and Hindi descriptions, and to run a user study that measures how many faculty visits the portal saves over one project cycle.

Acknowledgment

The authors thank their guide, Dr. D.B.K. Kamesh, and the Department of Computer Science and Engineering, MLR Institute of Technology, for their guidance and support.

References

[1] S. Jumoke and O. I. Folashade, “A web-based final year students project duplication detection system,” *Int. J. Comput. Appl.* [Online]. Available: https://www.researchgate.net/publication/312523925

[2] “Academic project information management system,” in *Proc. IEEE Conf.*, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9544565

[3] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[4] “Plagiarism detection in students’ theses using the cosine similarity method.” [Online]. Available: https://www.academia.edu/65392049

[5] V. K. Vani and D. Gupta, “Unmasking text plagiarism using syntactic-semantic based natural language processing techniques: Comparisons, analysis and challenges,” *Inf. Process. Manage.*, vol. 54, 2018.

[6] A. Amirzhanov, C. Turan, and A. Makhmutova, “Plagiarism types and detection methods: A systematic survey of algorithms in text analysis,” *Front. Comput. Sci.*, vol. 7, 2025.

[7] A. Barrón-Cedeño, “On the mono- and cross-language detection of text re-use and plagiarism,” in *Proc. 33rd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2010.

[8] “AI based plagiarism checking: Ease of use and applicable system for teachers to find similarities in students’ assessments,” in *Proc. IEEE Conf.*, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10046379

[9] “Fast and reliable plagiarism detection system,” in *Proc. IEEE Conf.* [Online]. Available: https://ieeexplore.ieee.org/document/4417860

[10] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[11] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *Proc. NeurIPS*, 2020.

[12] Turnitin, “Turnitin Core API (iThenticate v2 API) documentation.” [Online]. Available: https://developers.turnitin.com

[13] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, “OpenID Connect Core 1.0,” OpenID Foundation, 2014.

[14] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-hashing for message authentication,” RFC 2104, Feb. 1997.

[15] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.